



Institute of Information Technology,

University of Dhaka

Software Requirements Specification and Analysis

Software Project Lab - III [SE-801]

Project Name: RapidEye

Supervised By:

Dr. Ahmedul Kabir,

Associate Professor,

Institute of Information Technology,

University of Dhaka.

Submitted By:

Mohammad Momenuzzaman (BSSE 1227)

_____

Supervisor’s Signature

Letter of Transmittal

26 November, 2024

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Technical Report Submission for SPL 3

Sir,

I am pleased to submit the technical report for my project, “RapidEye: Smart Video Monitoring and Alert System for Abnormal and High-Speed Events”, as part of our Software Project Lab 3. This report provides an in-depth overview of the project's technical aspects, encompassing its design, implementation, and testing phases.

While I have made every effort to ensure the report is thorough and detailed, I sincerely welcome your feedback and suggestions for further improvement.

Thank you for your time and consideration.

Sincerely yours,

Mohammad Momenuzzaman

Roll: BSSE 1227

Institute of Information Technology

University of Dhaka

_____

Supervisor’s Signature

Acknowledgment

I am deeply grateful to the Institute of Information Technology for granting me the incredible opportunity to work on “RapidEye: Smart Video Monitoring and Alert System for Abnormal and Rapid Events”. I would like to express my heartfelt gratituude to my supervisor, Dr. Ahmedul Kabir, Associate Professor at the Institute of Information Technology, University of Dhaka, for his invaluable guidance in preparing this analysis document, and for his continued support and feedback on my activities related to the project. Finally, I extend my sincere thanks to my classmates, whose constant support and valuable insights have been immensely helpful throughout this process.

Abstract

The document outlines software requirements, specifications, architectural design, and testing of *“RapidEye: Smart Video Monitoring and Alert System for Abnormal and High-Speed Events.”* This project intends to analyze real-time and prerecorded video footage, identifying rapid or unusual events through ML algorithms. It integrates cutting-edge video processing techniques to detect anomalies, trigger alerts, and provide actionable insights.

The system leverages machine learning and computer vision to process video data, extract key patterns, and highlight significant events. Additionally, it offers user-friendly visualizations and customizable alerts to enhance understanding and usability. This application is deemed necessary for surveillance purposes, emergency response units, and organizations seeking to automate and optimize video monitoring for safety, security, and computational efficiency.

Table of Contents

Table of Contents **4**
Motivation **7**
1. Introduction **8**
1.1 Purpose 8
1.2 Scope 8
1.3 Project Timeline 9
1.4 Purpose of Document 9
2. Project Description **10**
2.1. Core Functionalities 10
2.1.1. Normal Requirements 10
2.1.2. Expected Requirements 10
2.1.3. Exciting Requirements 11
2.2. Usage Scenarios 11
2.1.1. Video Input Processing 11
2.2.2. Movement Detection and Analysis 11
2.2.3. Real-Time Alert Generation 12
2.2.4. Event Recording and Review 12
2.2.5. System Configuration 12
2.2.6. Multi-Camera Support 12
3. Scenario-Based Modeling **14**
3.1 Use Case Diagram 14
3.1.1 Level 0 15
Figure 1: Use Case Diagram Level 0 15
3.1.2 Level-1 16
Figure 2: Use Case Diagram Level 1 16
3.1.3 Level 1.1 17
Figure 3: Use Case Diagram Level 1.1 17
3.1.4 Level 1.1.5 18
Figure 4: Use Case Diagram Level 1.1.5 18
3.1.5 Level 1.2 19
Figure 5: Use Case Diagram Level 1.2 19
3.1.6 Level 1.3 20
Figure 6: Use Case Diagram Level 1.3 20

3.1.7 Level 1.4 21
Figure 7: Use Case Diagram Level 1.4 21
5. Class Based Modeling 22
5.1 Final List of Classes 22
5.2 Class Card 23
5.2.1 Video 23
5.2.2 Movement 23
5.2.3 Alert 24
5.2.4 Zone 24
5.2.5 Operator 25
5.2.6 Settings 26
Figure 8: CRC Diagram 27
6. Software Design Architecture 28
6.1 Architectural Context Diagram 28
Figure 9: Architectural Context Diagram 29
6.2 Defining Archetypes 29
6.2.1 Pipe and Filter Architecture 29
Figure 10: Pipe and Filter Pattern for Video Processing 29
6.2.2 Event Driven Architecture 30
Figure 11: Event-Driven Architecture for Alert Management 30
6.2.3 Layered Architecture 31
Figure 12: Layered Architecture 31
6.3 Refining the Architecture into components 32
6.3.1 Top level Components 32
Figure 13: Top Level Components 32
6.3.2 Instantiation 33
Figure 14: Instantiation of components to second level 33
6.4 Mapping Requirements to Software Architecture 34
7. Test Plan 35
7.1 High-Level Testing Goals 35
7.2 Test Cases 36
Conclusion 40

Motivation

In a world increasingly dependent on surveillance to achieve safety, security, and operational efficiency, the deficiencies of traditional monitoring systems are very conspicuous. It is increasingly difficult to maintain pace with the amount and intricacy of video information by mere observation and antiquated equipment; this leads to poor incident response times, incidents missed, and compromised decision-making.

With RapidEye, we intend to reinvent video monitoring through real-time analysis and the precise detection of abnormal and high-speed events.

This dynamic system goes beyond traditional surveillance, offering enhanced efficiency, scalability, and ease of use. Whether for public safety, industrial operations, or emergency response, RapidEye empowers organizations to act swiftly, decisively, and effectively. It is a bold leap forward in video analytics, reshaping how we monitor, respond to, and manage critical events in an increasingly fast-paced world.

1. Introduction

This chapter provides an overview of the software requirements and specifications for *RapidEye*. It outlines the purpose, scope, and target audience of the project, offering a clear foundation for development.

1.1 Purpose

The objectives of this document are as follows:

- To provide a detailed outline of the software requirements for *RapidEye*, a smart video monitoring and alert system for detecting abnormal and rapid events.
- To clearly define functional and non-functional requirements, ensuring a common understanding among all stakeholders.
- To establish a strong foundation for the development process, aligning expectations and enabling effective execution.

1.2 Scope

The scope of *RapidEye* is as follows:

- The system will support real-time and prerecorded video analysis.
- It will detect and generate alerts for abnormal or rapid events in video footage.
- Users can customize alert parameters and access detailed event logs.
- The application will include data visualization to enhance user understanding.
- Features like user management and data retention will provide a seamless and tailored experience.

1.3 Project Timeline

TASKS	WEEK 1	WEEK 2	WEEK 3	WEEK 4	WEEK 5	WEEK 6	WEEK 7	WEEK 8	WEEK 9	WEEK 10	WEEK 11	WEEK 12	WEEK 13	WEEK 14	WEEK 15	WEEK 16	WEEK 17	WEEK 18	WEEK 19
Project Selection																			
Project Initialization																			
SRS Documentation																			
Project Design																			
Project Development																			
Project Testing																			
Report writing & Submission																			

Figure-1: Timeline for RapidEye

1.4 Purpose of Document

The purposes of this document are as follows:

- Identify and analyze the requirements for *RapidEye*.
- Design the architectural framework for the system.
- Develop a comprehensive test plan to ensure system reliability.
- Outline the methodology for implementing and testing key features.
- Streamline development processes to minimize effort and maximize efficiency.
- Enhance clarity and understanding of the project's goals and deliverables.

2. Project Description

The following outlines the overview of the core functionalities and objectives of RapidEye. The conceptualization began with a review of surveillance and security systems, particularly focusing on the need for automated threat detection in real-time video streams. The project's scope and requirements were defined through consultations with security professionals and potential end-users, ensuring alignment with practical security monitoring needs.

2.1. Core Functionalities

RapidEye provides an AI-powered solution for real-time detection of suspicious activities and potential threats through video surveillance analysis. The functionalities are grouped into three categories:

2.1.1. Normal Requirements

Video Processing: RapidEye must process video streams (both real-time and recorded) to detect and track human movements.

Movement Analysis: The system must analyze detected movements to identify rapid or suspicious actions using computer vision and pose estimation.

Multiple Person Tracking: The system should accurately track and analyze multiple individuals simultaneously in the video frame.

Alert Generation: Real-time alerts must be generated when suspicious movements or interactions are detected.

2.1.2. Expected Requirements

Detection Accuracy: The AI models should maintain high accuracy in detecting and analyzing human movements.

Performance Optimization: The system must operate efficiently on CPU hardware for widespread deployment capability.

Robust Tracking: The system should maintain accurate tracking even with partial occlusions or challenging angles.

2.1.3. Exciting Requirements

Interaction Analysis: Detection of interactions between multiple individuals and assessment of potential threats.

Event Recording: Automatic recording and storage of detected suspicious events for future reference.

Customizable Sensitivity: Adjustable thresholds for movement detection and alert generation based on deployment context.

Weapon Detection: Future capability to detect weapons and other threatening objects.

2.2. Usage Scenarios

RapidEye functions as a video surveillance system enhanced with AI capabilities for automated threat detection. The usage scenarios are as follows:

2.1.1. Video Input Processing

The system accepts video input from surveillance cameras and processes each frame to detect human presence using object detection.

2.2.2. Movement Detection and Analysis

For each detected person, the system:

- Applies pose estimation using MediaPipe
- Tracks key body points and analyzes movement patterns

- Calculates movement speeds and identifies rapid or suspicious movements
- Detects interactions between multiple individuals

2.2.3. Real-Time Alert Generation

When suspicious activities are detected, the system:

- Generates immediate alerts with relevant details
- Marks the concerning individuals or interactions in the video feed
- Provides movement speed and interaction proximity data

2.2.4. Event Recording and Review

The system maintains a record of detected events, allowing security personnel to:

- Review marked footage of suspicious activities
- Analyze movement patterns and interaction data
- Export relevant clips for documentation or further analysis

2.2.5. System Configuration

Security administrators can:

- Adjust sensitivity thresholds for movement detection
- Configure alert parameters
- Customize detection zones and monitoring priorities
- Manage recording and storage settings

2.2.6. Multi-Camera Support

RapidEye processes feeds from multiple cameras while:

- Maintaining independent analysis for each stream
- Correlating events across different cameras

- Providing a unified interface for monitoring and alerts

This structured approach ensures that RapidEye fulfills its goal of providing an efficient, automated, and reliable system for security monitoring and threat detection.

3. Scenario-Based Modeling

Scenario-based modeling is a method used in software development to create models that represent system behavior under various real-world conditions. For RapidEye, the scenarios focus on user interactions and how the app handles different events, enabling developers to identify potential issues and design solutions effectively.

3.1 Use Case Diagram

A use case diagram is a key visualization tool in software engineering that illustrates how a system interacts with external entities, or actors, to accomplish specific tasks or objectives. It provides a clear representation of the relationships between use cases, which depict the system's functionalities, and actors, which represent external users, devices, or other systems.

Typically created during the requirements gathering phase, a use case diagram helps identify the core functionalities of the system and how users will interact with them. These requirements are then translated into a graphical format using software modeling tools, ensuring a structured and organized view of the system's intended behavior.

By highlighting the connections between users and system functionalities, use case diagrams offer a concise overview of how the system operates, aiding in communication and alignment among stakeholders, designers, and developers.

3.1.1 Level 0

Name: RapidEye Surveillance System

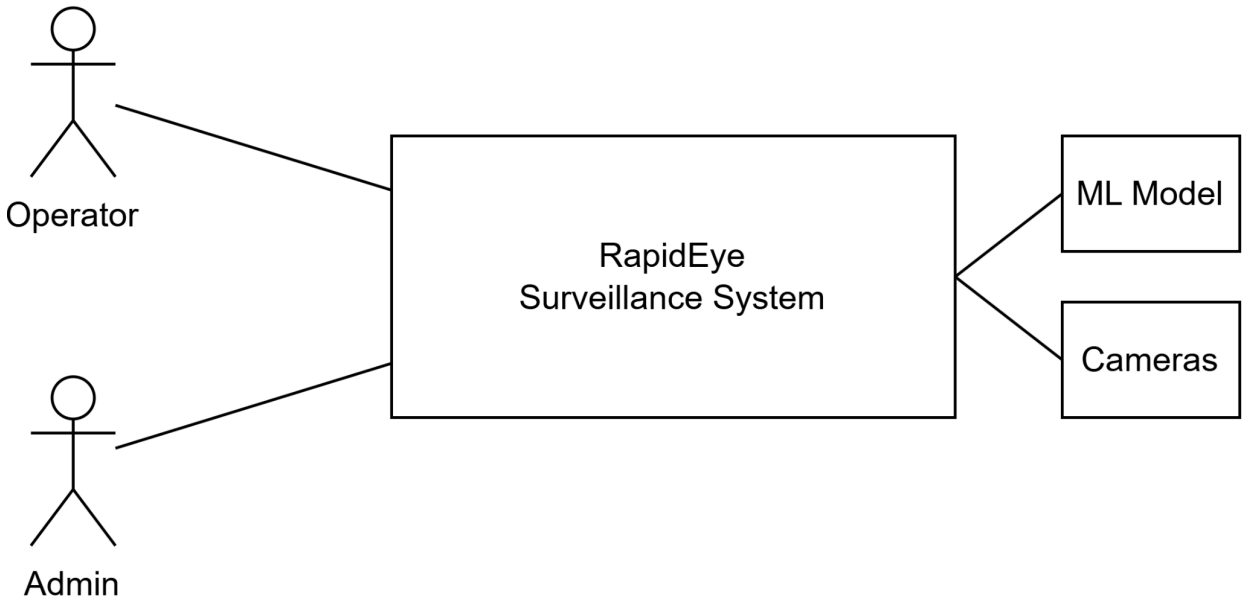


Figure 1: Use Case Diagram Level 0

Primary Actors:

- Operator
- Admin

Secondary Actors:

- ML Model
- Cameras

Both actors interact directly with the core RapidEye system, establishing the highest level view of system interaction.

3.1.2 Level-1

Name: RapidEye System

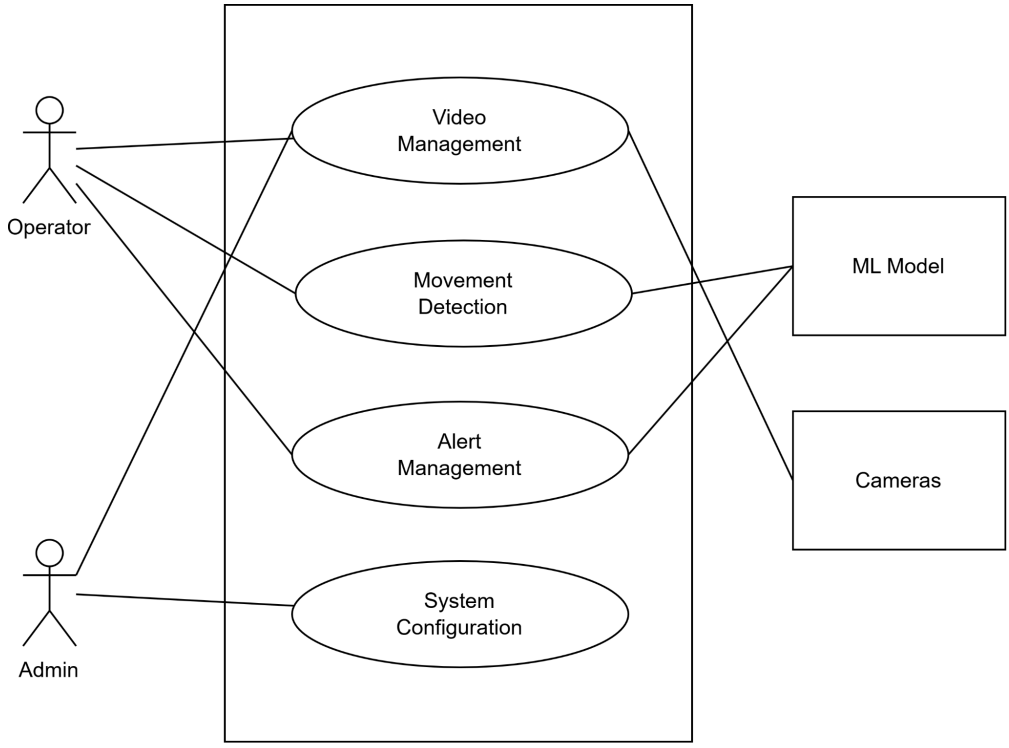


Figure 2: Use Case Diagram Level 1

Primary Actors:

- Operator (interacts with Video Management, Movement Detection, Alert Management)
- Admin (interacts with System Configuration and Video Management)

Secondary Actors:

- ML Model
- Cameras

Main use cases include:

- Video Management: Control and management of video feeds
- Movement Detection: Detection and analysis of suspicious movements
- Alert Management: Handling system alerts and notifications
- System Configuration: Managing system settings and parameters

3.1.3 Level 1.1

Name: Video Management

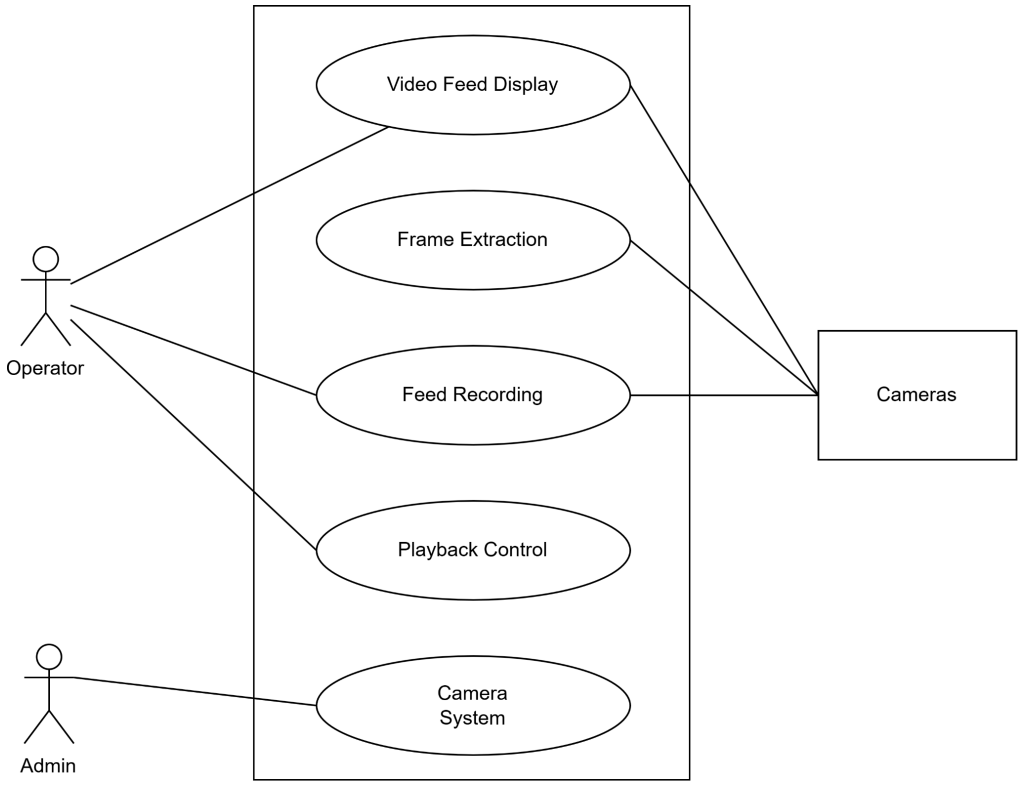


Figure 3: Use Case Diagram Level 1.1

Primary Actors:

- Operator (interacts with first 4 use cases)
- Admin (interacts with Camera System)

Secondary Actors: Cameras

Use cases include:

- Video Feed Display: Real-time display of surveillance feeds
- Frame Extraction: Processing video frames for analysis
- Feed Recording: Storing video footage
- Playback Control: Managing recorded video playback
- Camera System: Hardware management and control

3.1.4 Level 1.1.5

Name: Camera System

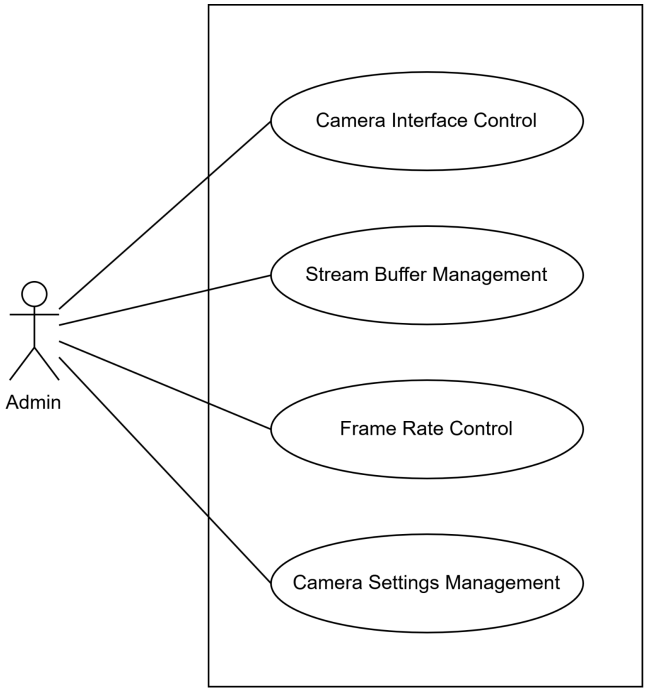


Figure 4: Use Case Diagram Level 1.1.5

Primary Actor: Admin

Use cases include:

- Camera Interface Control: Managing camera connections and interfaces
- Stream Buffer Management: Handling video stream buffering
- Frame Rate Control: Managing video frame rates

- Camera Settings Management: Configuring camera parameters

3.1.5 Level 1.2

Name: Movement Detection

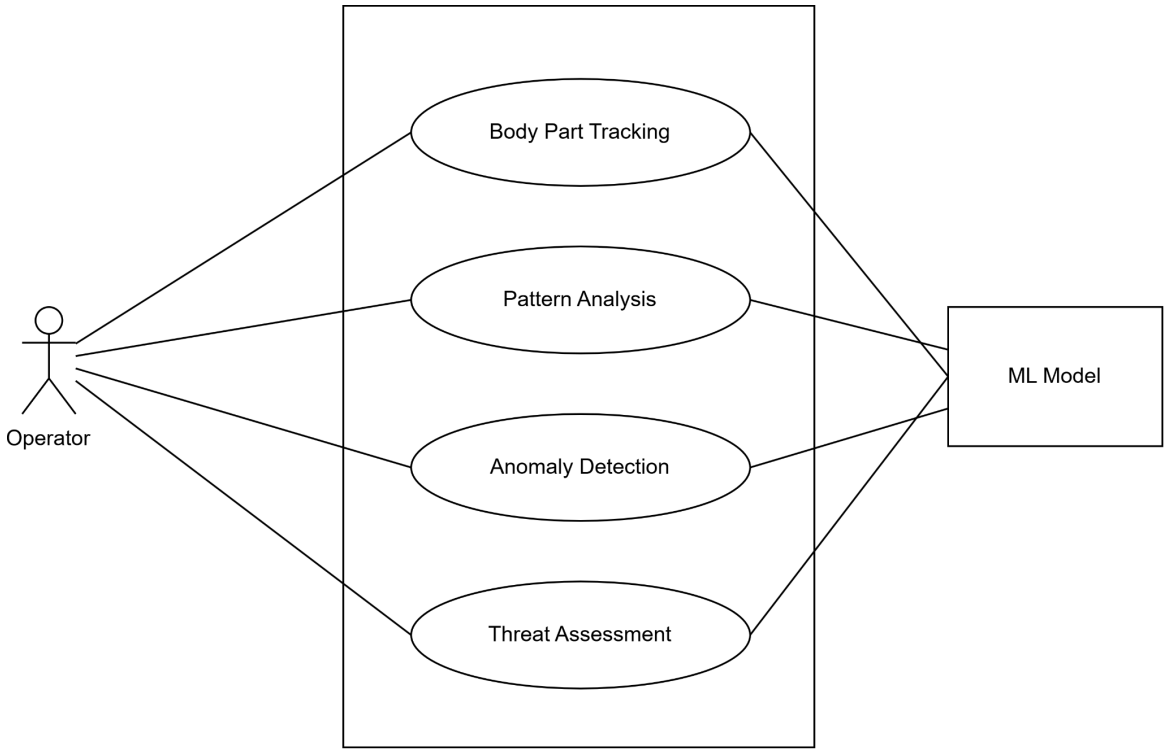


Figure 5: Use Case Diagram Level 1.2

Primary Actor: Operator

Secondary Actor: ML Model

Use cases include:

- Body Part Tracking: Tracking human body parts and movements
- Pattern Analysis: Analyzing movement patterns
- Anomaly Detection: Identifying unusual movements
- Threat Assessment: Evaluating potential security threats

3.1.6 Level 1.3

Name: Alert Management

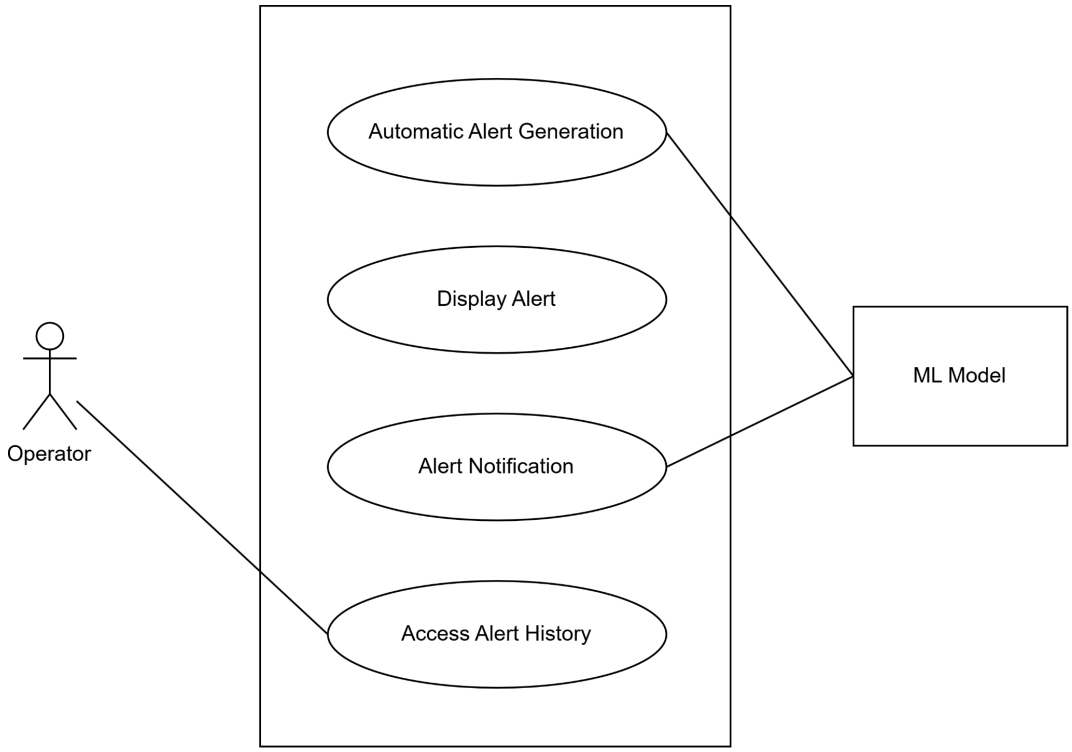


Figure 6: Use Case Diagram Level 1.3

Primary Actor: Operator

Secondary Actor: ML Model

Use cases include:

- Automatic Alert Generation: Creating alerts based on detections
- Display Alert: Showing alerts in the interface
- Alert Notification: Sending notifications to operators
- Access Alert History: Reviewing past alerts

3.1.7 Level 1.4

Name: System Configuration

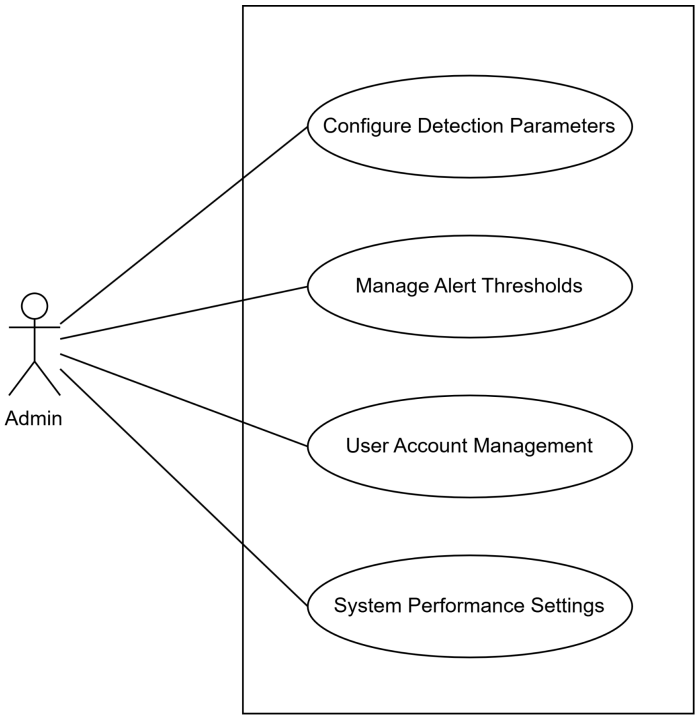


Figure 7: Use Case Diagram Level 1.4

Primary Actor: Admin

Use cases include:

- Configure Detection Parameters: Setting up detection sensitivity and criteria
- Manage Alert Thresholds: Configuring alert triggering conditions
- User Account Management: Managing system users and permissions
- System Performance Settings: Optimizing system performance parameters

5. Class Based Modeling

Class-based modeling establishes the static structure of objects within a system, defining the overall framework of the system. A class model specifies the attributes and methods of objects in each class and outlines their interactions and collaborations across different classes. This modeling approach comprises several key components, including classes and objects, attributes, methods, class-responsibility-collaborator (CRC) models, collaboration diagrams, and packages.

5.1 Final List of Classes

First, I identified classes from the solution domain using general classifications such as system components, management units, and processing elements. Through analysis and refinement based on criteria like retained information, required services, multiple attributes, and essential requirements, I identified the following analysis classes:

1. Video
2. Movement
3. Alert
4. Zone
5. Operator
6. Settings

5.2 Class Card

5.2.1 Video

Class Name: Video	
Attributes	**Methods**
-SourceID -streamType (live/stored) -frames -recordingPath -timestamp	+startStream() +stopStream() +recordVideo() +playbackVideo() +extractFrames()
Responsibilities	**Collaborator Class**
- Managers video input stream - Handles frame processing - Controls Recording and Playback - Provides frame access for analysis	- Movement - Zone

5.2.2 Movement

Class Name: Movement	
Attributes	**Methods**
-trackingID -personCount -movementSpeed -patternType -detectionTime	+detectPerson() +trackMovement() +analyzeBehaviour() +calculateSpeed() +validatePattern()
Responsibilities	**Collaborator Class**
- Track and identify human presence - Calculate movement speeds and patterns - Identify suspicious behaviour - Process multiple simultaneous tracks	- Video - Zone - Alert

5.2.3 Alert

Class Name: Alert	
Attributes	**Methods**
-alertID -alertType -severity -status -timestamp -description	+generateAlert() +updateStatus() +notifyOperator() +logAlert() +retrieveAlertHistory()
Responsibilities	**Collaborator Class**
- Generate Alerts for suspicious activities - Manage Alert Lifecycle - Maintain Alert History - Notify Relevant Operators	- Movement - Zone - Operator

5.2.4 Zone

Class Name: Zone	
Attributes	**Methods**
-zoneID -boundaries -sensitivity -activeStatus -alertThreshold -description	+defineZone() +updateBoundaries() +setSensitivity() +validateActivity() +getZoneStatus()
Responsibilities	**Collaborator Class**

- Define monitoring areas - Manage Zone Parameters - Track zone specific activities - Control detection sensitivity	- Movement - Video - Alert

5.2.5 Operator

Class Name: Operator	
Attributes	**Methods**
-operatorID -accessLevel -status -astLogin -currentSession	+login() +handleAlert() +modifySettings() +generateReport() +manageZones()
Responsibilities	**Collaborator Class**
- Monitor System Activities - Respond to alerts - Manage system configurations - Control access permissions	- Alert - Zone - Settings

5.2.6 Settings

Class Name: Settings	
Attributes	**Methods**
-configID -detectionParams -alertThresholds -systemPreferences -performanceMetrics	+updateConfig() +loadDefaults() +validateSettings() +applyChanges() +backupConfig()
Responsibilities	**Collaborator Class**
- Manage System Configurations - Control Detection Parameters - Handle performance settings - Maintain System preferences	- Operator - Zone - Alert

So based on the class cards, we can create the following table to summarise it:

Class Name	Collaborator Class
Video	Movement, Zone
Movement	Video, Alert, Zone
Alert	Movement, Zone, Operator
Zone	Video, Movement, Alert
Operator	Alert, Zone, Settings
Settings	Operator, Zone, Alert

The CRC Diagram is given below.

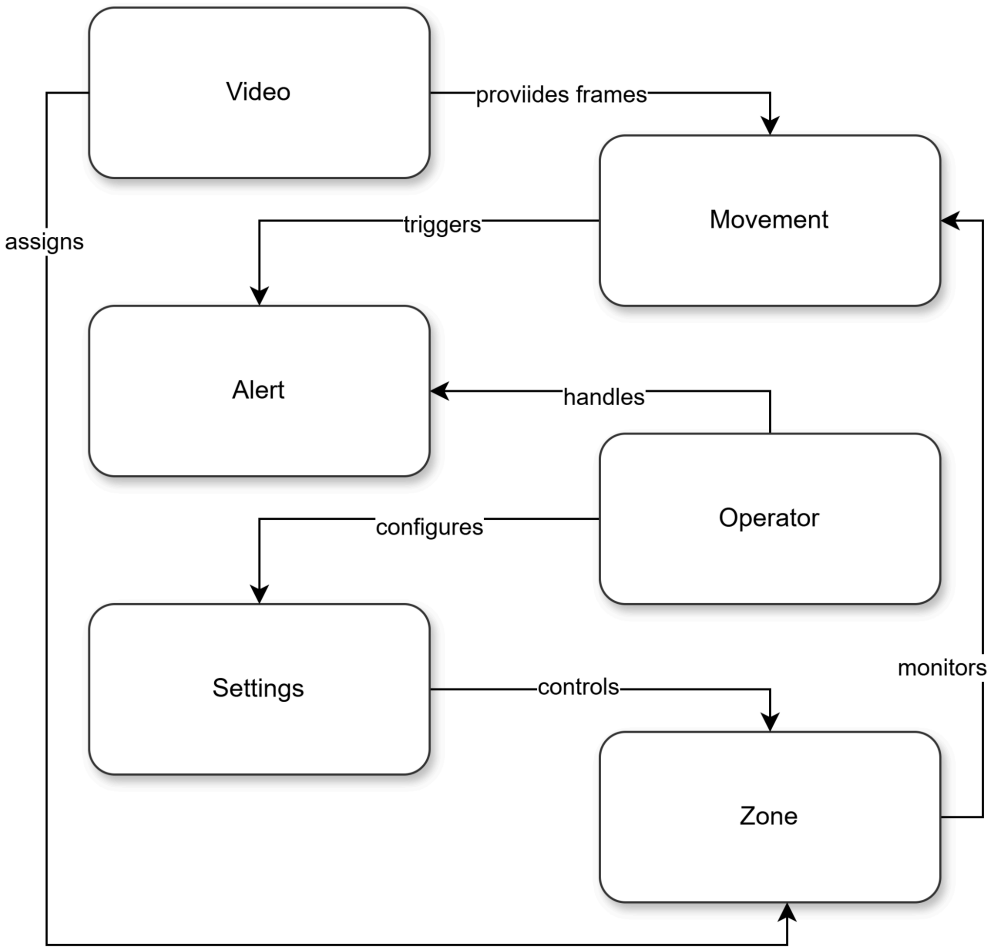


Figure 8: CRC Diagram

6. Software Design Architecture

When starting architectural design, it is essential to place the software in context by identifying the external entities it interacts with and the nature of those interactions. This information is typically derived from the requirements model and other data gathered during requirements engineering. Architectural design outlines the structure of the data and program components needed to develop a computer-based system. The process involves four key steps.

6.1 Architectural Context Diagram

At the architectural design level, an architectural context diagram models how software interacts with external entities that communicate with the target system using interfaces.

Superordinate systems:

- Video Management System

Subordinate systems:

- Movement Detection Engine
- Pose Estimation Service
- Pattern Recognition Service

Peer-level systems:

- None for our system

Actors:

- Security Operators
- System Administrators

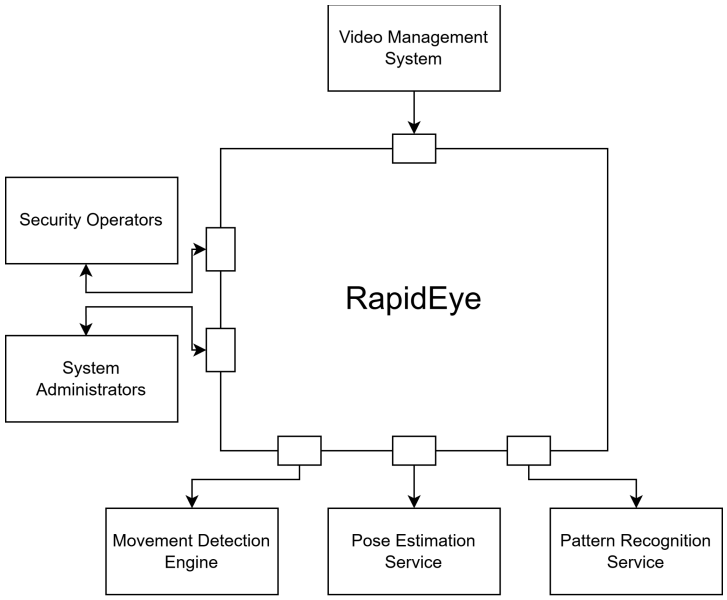


Figure 9: Architectural Context Diagram

6.2 Defining Archetypes

Our system employs three main architectural patterns that work together to create a robust surveillance system.

6.2.1 Pipe and Filter Architecture

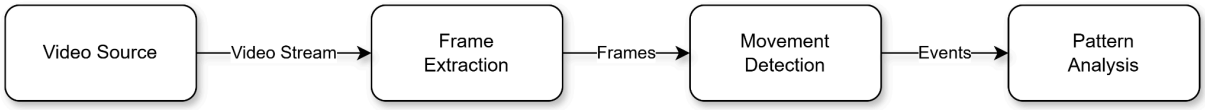


Figure 10: Pipe and Filter Pattern for Video Processing

Sequential processing of video data through specialized components:

- Video Source → Frame Extraction → Movement Detection → Pattern Analysis
- Each component performs a specific transformation on the data
- Components are connected through data streams

6.2.2 Event Driven Architecture

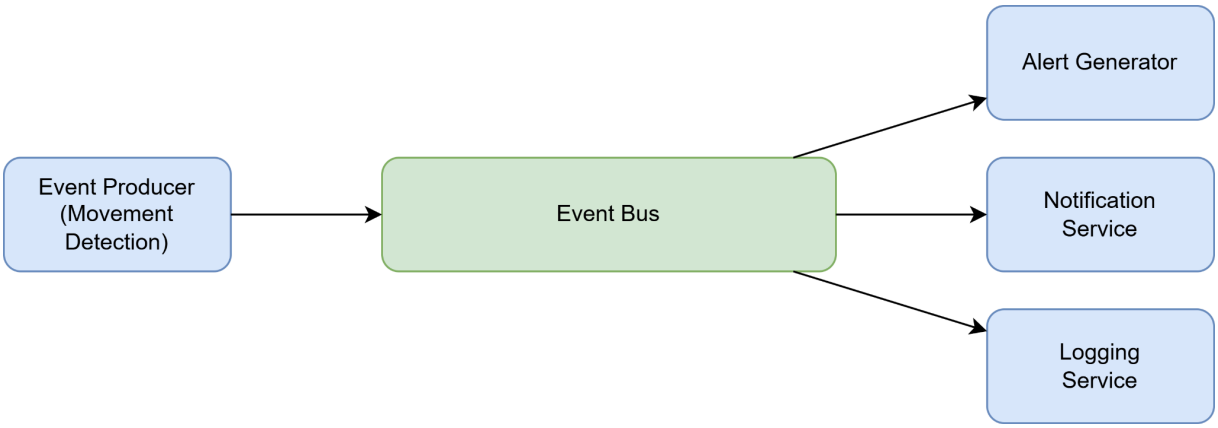


Figure 11: Event-Driven Architecture for Alert Management

An event-driven approach for handling detected events and alerts:

- Event Producer generates movement detection events
- Central Event Bus routes events to appropriate handlers
- Multiple consumers (Alert Generator, Notification Service, Logging Service) process events independently

6.2.3 Layered Architecture

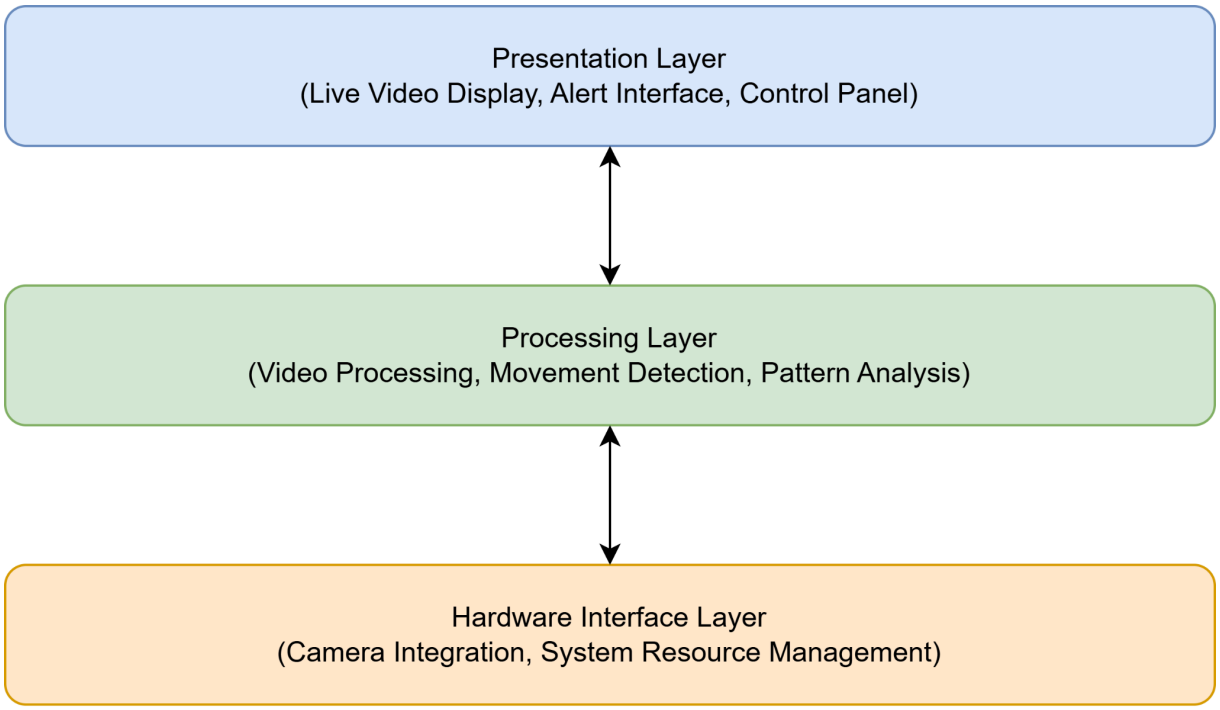


Figure 12: Layered Architecture

This pattern divides the system into three distinct layers:

- Presentation Layer: Handles user interaction through video display, alerts, and control interfaces
- Processing Layer: Manages core functionality including video processing and detection algorithms
- Hardware Interface Layer: Manages system resources and camera integration

6.3 Refining the Architecture into components

6.3.1 Top level Components

Top level components have been illustrated below. Top level components give us a very high level view of the system, and requires further refinement, which is done in subsequent steps. We limit the top level components to 2 layers because our system is inherently not very complicated, as well as facilitating understanding of the system from a top-down perspective.

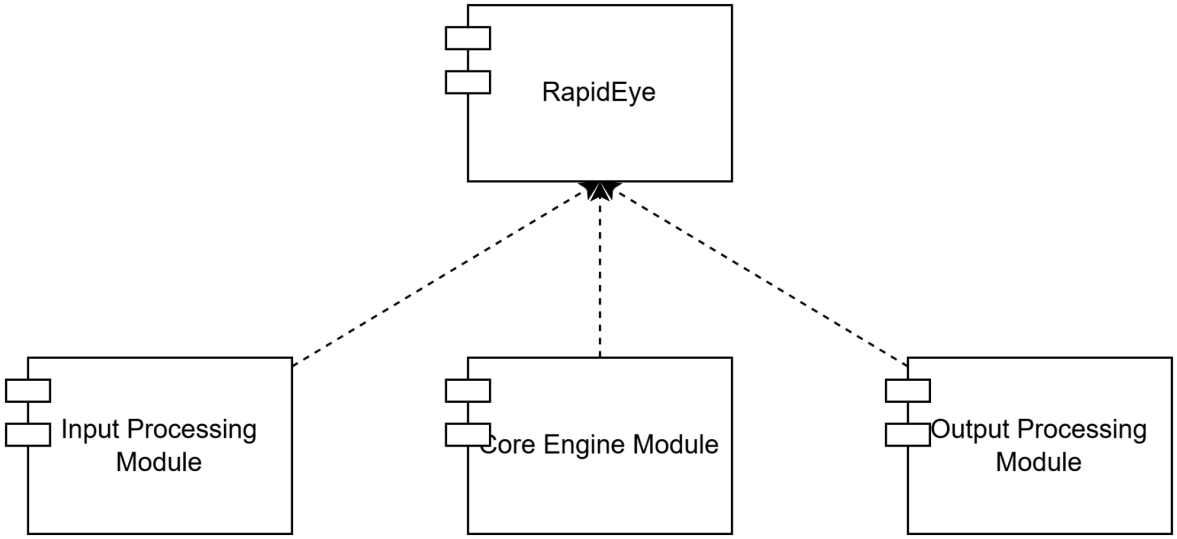


Figure 13: Top Level Components

6.3.2 Instantiation

The components identified at the Top Level are broad and require further refinement into smaller, more detailed components. However, simply assembling a collection of components does not result in a coherent design. To create a maintainable and effective system, architecture is applied to these components. This ensures the structure and components are tailored to address a specific problem, demonstrating their suitability and effectiveness.

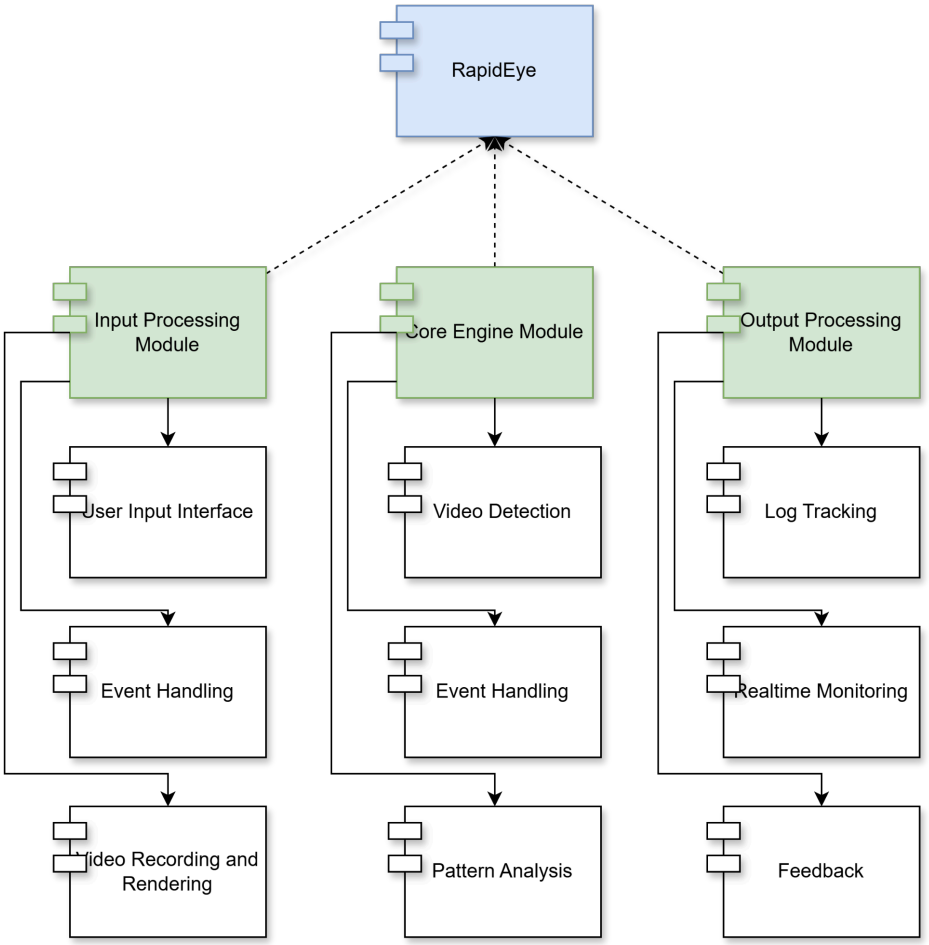


Figure 14: Instantiation of components to second level

6.4 Mapping Requirements to Software Architecture

Functional Requirement	Mapped Component(s)
Video Processing: Real-time and stored video analysis	Processing System (Input Interface, Video Recording and Rendering, Event Handling)
Movement Detection: Track and analyze human movements	Video Analysis & AI (Video Detection, Movement Analysis, Pattern Analysis)
Multiple Person Tracking: Track multiple individuals simultaneously	Video Analysis & AI (Movement Analysis, Pattern Analysis)
Alert Generation: Generate real-time alerts for suspicious movements	Monitoring and Tracking (Real-Time Monitoring, Feedback Analysis)
Performance Optimization: Efficient CPU hardware operation	Cross-cutting across all components with hardware resource management
Event Recording: Automatic recording of detected events	Processing System (Video Recording) + Monitoring and Tracking (Tracking by Logs)
Interaction Analysis: Detect and assess potential threats from interactions	Video Analysis and AI (Pattern Analysis) + Monitoring and Tracking (Tracking by Logs)

7. Test Plan

7.1 High-Level Testing Goals

Testing goals from a high level can be classified as the following:

1. Validate Video Processing Accuracy

   - Verify frame extraction and processing

   - Test movement detection precision

   - Validate pattern recognition accuracy

2. Verify Alert System Functionality

   - Test alert generation timing

   - Validate notification delivery

   - Verify alert classification accuracy

3. Evaluate System Performance

   - Test multi-camera processing capability

   - Measure system response times

   - Validate resource utilization

4. Assess Integration Points

- Test camera interface compatibility
- Verify external system communications
- Validate data storage and retrieval

7.2 Test Cases

TC01: Video Input Processing
Objective: Verify video stream handling
Pre-conditions: System running, camera connected
Test Steps:
1. Connect multiple video sources
2. Verify frame capture rate
3. Check video quality retention
4. Validate multi-stream processing
Expected Results: Smooth video processing without frame drops

TC02: Movement Detection
Objective: Validate movement detection accuracy
Pre-conditions: Video input active
Test Steps:
1. Simulate various movement patterns
2. Test detection in different lighting
3. Verify detection zones functionality
4. Measure false positive rate
Expected Results: >75% detection accuracy with <25% false positives

TC03: Alert Generation
Objective: Test alert system functionality
Pre-conditions: Movement detection active
Test Steps:
1. Trigger various alert conditions
2. Verify alert classification
3. Check notification delivery
4. Validate alert history recording
Expected Results: Real-time alert generation with correct classification

TC04: System Performance
Objective: Evaluate system under load
Pre-conditions: Multiple cameras active
Test Steps:
1. Process multiple video streams
2. Monitor CPU/memory usage
3. Test system responsiveness
4. Measure alert generation latency
Expected Results: Stable performance within resource limits

TC05: User Interface
Objective: Verify UI functionality
Pre-conditions:System operational
Test Steps:
1. Test all navigation paths
2. Verify display layouts
3. Check real-time updates
4. Validate user input handling
Expected Results: Responsive interface

TC06: Configuration Management
Objective: Test system configuration features
Pre-condition: Admin access available
Test Steps:
1. Modify detection parameters
2. Update camera settings
3. Configure alert rules
4. Test user management
Expected Results: Accurate application of all configurations

TC07: Integration Testing
Objective: Validate system integration points
Pre-conditions: All subsystems operational
Test Steps:
1. Test camera integration
2. Verify storage system
3. Check notification delivery
4. Validate external interfaces
Expected Results: Seamless interaction between components

TC08: Recovery Testing
Objective: Verify system recovery capabilities
Pre-conditions: System running normally
Test Steps:
1. Simulate power failure
2. Test network interruption
3. Check data recovery
4. Verify service restoration
Expected Results: Automatic recovery with data integrity

TC09: Security Testing
Objective: Validate system security
Pre-conditions: System fully configured
test Steps:
1. Test access controls
2. Verify data encryption
3. Check audit logging
4. Test intrusion detection
Expected Results: All security measures functioning properly

TC10: Long-Term Stability
Objective: Verify system stability over time
Pre-conditions: System in production environment
Test Steps:
1. Run continuous operation
2. Monitor resource usage
3. Check for memory leaks
4. Verify data management
Expected Results: Stable long-term operation

Conclusion

The RapidEye system combines real-time video surveillance with intelligent threat detection through a modular and scalable architecture. Through its three main components - Processing System, Video Analysis & AI, and Monitoring & Tracking - the system effectively handles video processing, movement detection, and alert generation.

The architectural patterns employed (layered, event-driven, and pipe-filter) ensure robust performance while maintaining flexibility for future enhancements. The comprehensive testing plan validates both functional requirements and system reliability.

Moving forward, RapidEye can be extended with additional AI capabilities and support for diverse camera systems, further strengthening its role in automated surveillance.